Static website generator

Calepin can build a Typst document directory as a static website. This is the same workflow used to render this documentation site: source `.typ` files live in `docs/`, and the generated `.html`, `.pdf`, and `sitemap.xml` files are written back into `docs/` for GitHub Pages.

This page covers the build workflow. See also:

- Site configuration for `website.toml` settings
- Navigation and listings for the sidebar, page titles, and blog-style page listings
- Themes for HTML theme customization

Create a website

Use `calepin new website` to scaffold a minimal site:

```
calepin new website
```

This creates:

- `website.toml`
- `docs/index.typ`
- `docs/404.typ`

Build pages

Compile a website directory with the same `compile` command used for a single Typst document:

```
calepin compile docs public --config website.toml
```

When the input path is a directory, `--config website.toml` is required. The first positional argument is the website source directory. The optional second positional argument is the website output directory.

For GitHub Pages publishing from `docs/`, build in place by passing `docs` as both the input and output directory:

```
calepin compile docs docs --config website.toml
```

If the output directory is omitted, *Calepin* writes output beside the source files:

```
calepin compile docs --config website.toml
```

By default, each `.typ` page produces two outputs:

- `page.html`
- `page.pdf`

PDF rendering can be disabled for the whole site in `website.toml`:

```
pdf = false
```

Individual pages can override the site setting with a `pdf` entry in their `<website-metadata>` metadata. This is useful to skip the PDF for a few heavy pages, or to render PDFs only for selected pages when the site default is off:

```
#metadata((pdf: false)) <website-metadata>
```

Use `--format html` to write only HTML, regardless of configuration and page metadata:

```
calepin compile docs public --config website.toml --format html
```

Directory website builds do not support `--format pdf`, `--format png`, or `--format svg`; those formats are for single-document `calepin compile`.

Website builds use the same preprocess cache as single-document compilation. After a successful page build, *Calepin* writes a fingerprint next to the page's `results.json`. Later builds reuse that file when the chunk code, parameters, execution options, configured tools, and source-relative cache paths match. This means `make serve` does not need to re-execute expensive chunks in pages such as `example.typ` when nothing relevant changed.

Generated outputs are tracked in `.calepin/website-manifest.json`. Later builds use that manifest to remove stale generated files when pages are deleted or renamed. Full builds also scan the output directory for unexpected generated `.html` and `.pdf` files, while preserving static assets in directories such as `assets/`, `.calepin/`, `.git/`, `target/`, `node_modules/`, and `.venv/`.

Watch changes

During development, watch the website source directory:

```
calepin watch docs docs --config website.toml
```

As with `compile`, the first positional argument is the source directory and the optional second positional argument is the output directory. The initial watch build renders the whole site. After that, existing `.typ` page edits go through a fast xxh3 fingerprint lookup:

- If the changed page hash is new, *Calepin* rebuilds only that page.
- If the file was touched but the hash is unchanged, *Calepin* skips the rebuild.
- If navigation or page metadata changed after an incremental rebuild, *Calepin* falls back to a full rebuild.

Calepin also falls back to a full rebuild for changes that can affect more than one page:

- `website.toml`
- theme files
- assets
- new `.typ` pages
- removed `.typ` pages
- renamed `.typ` pages
- unknown or non-page inputs

Watch and serve locally

To rebuild and serve in one command:

```
calepin watch docs docs --config website.toml --serve
```

The server injects a small reload script into HTML responses. Open browser pages poll the server and refresh after successful rebuilds. When a rebuild fails, the open pages display the build error as an overlay, and reload automatically once a later rebuild succeeds.

The default bind address is `127.0.0.1`, on the first free port starting at 8000. Use `--port` to pin a specific port; *Calepin* then fails instead of falling back to another port:

```
calepin watch docs docs --config website.toml --serve --port 8001
```

You can also bind another interface:

```
calepin watch docs docs --config website.toml --serve --host 0.0.0.0 --port 8001
```

Serve built files

Serve an already-built static directory with:

```
calepin serve docs
```

The static server:

- serves files from the requested directory
- maps directory requests to `index.html`
- rejects path traversal
- supports `GET` and `HEAD`
- guesses content types from file extensions
- picks the first free port from 8000 when `--port` is not given, and reports a clear error when a pinned port is already in use

Use `--host` and `--port` to change the bind address:

```
calepin serve docs --host 127.0.0.1 --port 8001
```

Current limitations

The website generator is intentionally small. It does not yet implement:

- dependency-aware rebuilds for imported shared `.typ` files
- drafts
- taxonomies
- pagination
- search indexes
- feeds
- robots.txt generation
- Sass or SCSS compilation
- link checking
- automatic browser opening

These can be added without changing the current command shape: `new`, `compile`, `watch`, and `serve`.